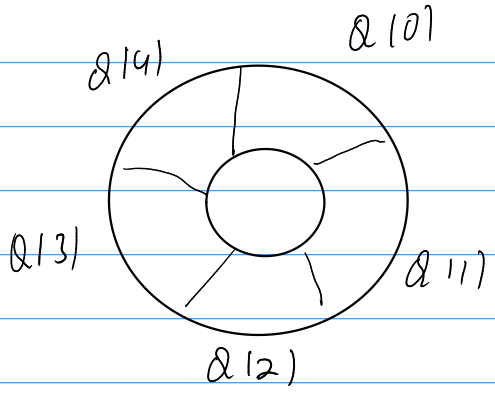
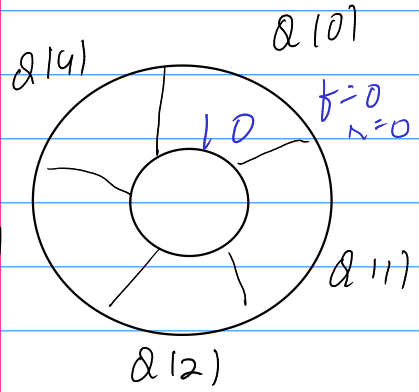
8/04

Circular Queue

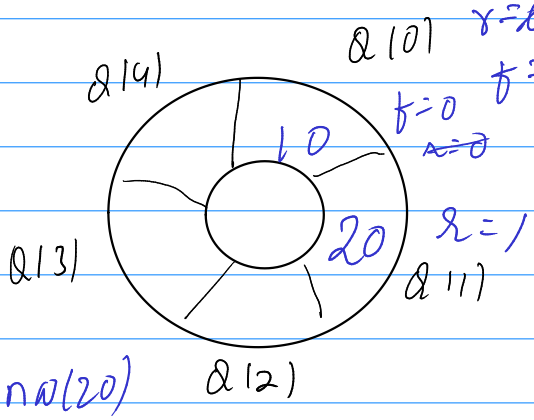
In Circular Queue after inserting last element or in last index rear is reset to -1



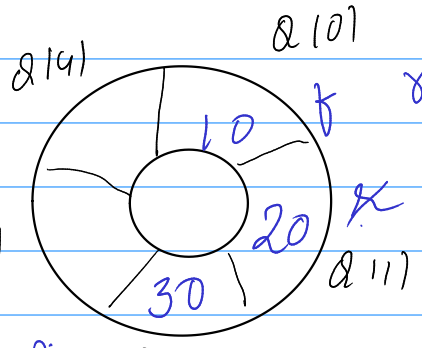
r = -1
f = -1



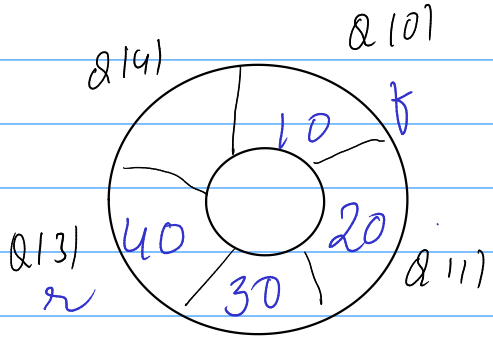
r = -1 0
f = -1 0
Enq(10)



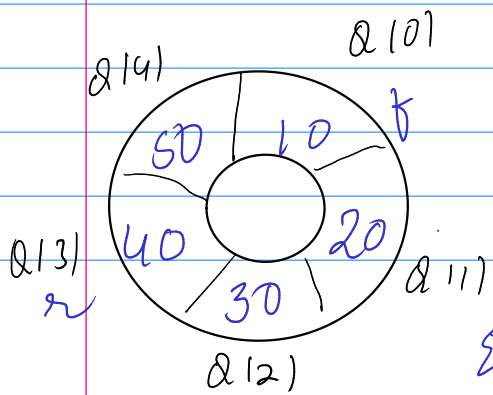
r=0 1
f=0
r=1
Enq(20)



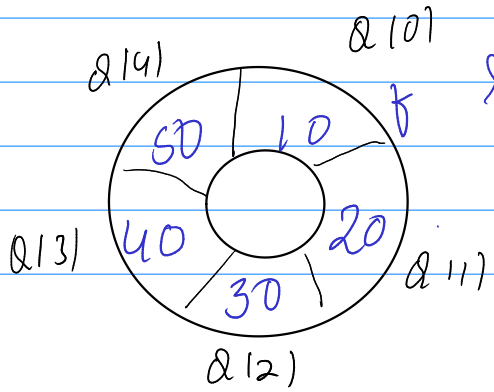
r=+2
f=0
r
Enq(30)



f=0
r=2 3
r
Enq(40)
r



f=0
r=3
4 -1
Enq(50)
r



r=-1
f=0

Applications of Queue

1. resource shared in a network.
   ex : printer
2. interrupts are executed in first come first serve
3. in os wait queue
4. scheduling algorithms
5. In some special files for transfering the data
   ex : fifo , pipe

Types of Queues:
1. linear queue
2. priority queue
3. double ended queue

Priority queue : In Priority queue data is deleted based on priority, i.e data with highest priority will deleted first

Double Ended Queue(dequeue) : In Dequeue
we can insert from rear end as well as front end
we can delete from front end as well as rear end

To represen the nonlinear or hierarchical representation we use nonlinear data datastructure

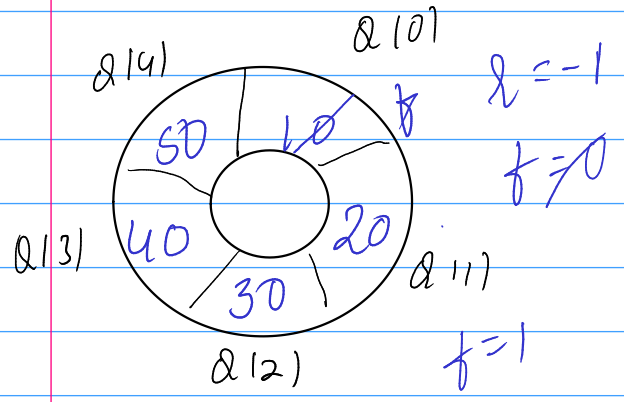
ex : trees
graphs

trees -----> Binary Search Tree

insertion
traversing
delete
search

if f==r+1 ----> queue full

```
if(f==r+1)
        Q full
else
{
if(front==-1)
        front++;
Q[++r]=data;
if(r==MAX-1)
        r=-1;
```

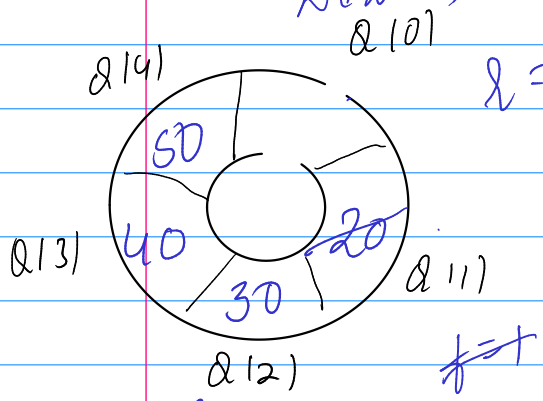


r=-1

f=0

f=1

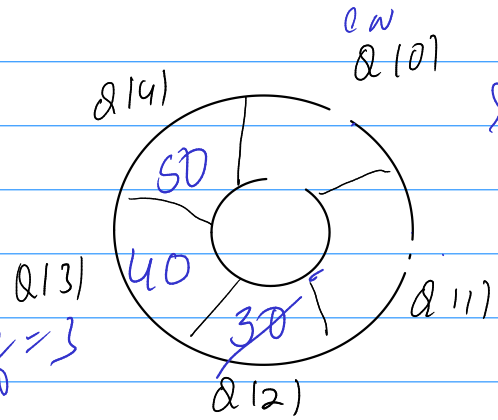
Deq()



r=-1

f=1

f=2 Deq()

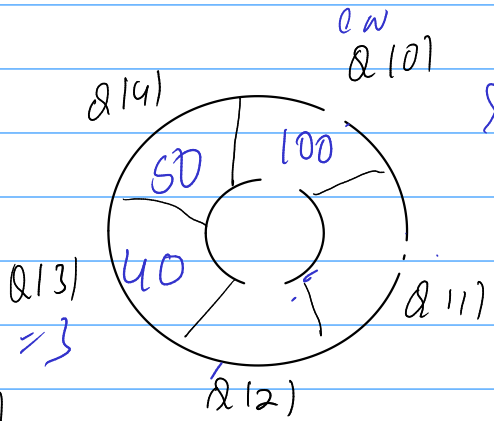


r=-1

f=3

f=2 Deq()

Enq(100)

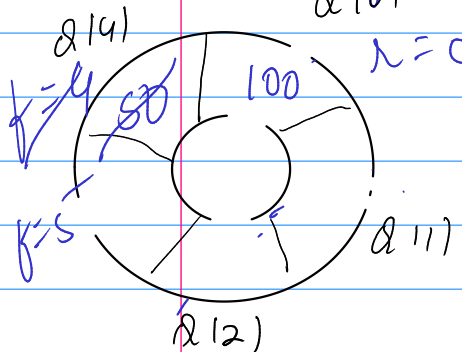


r=-1

r=0

f=4

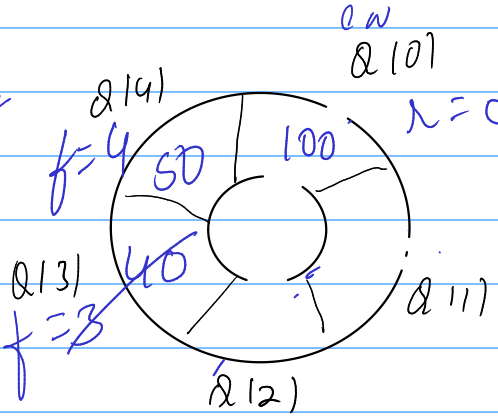
f=3



f=4

f=5

Deq()



r=0

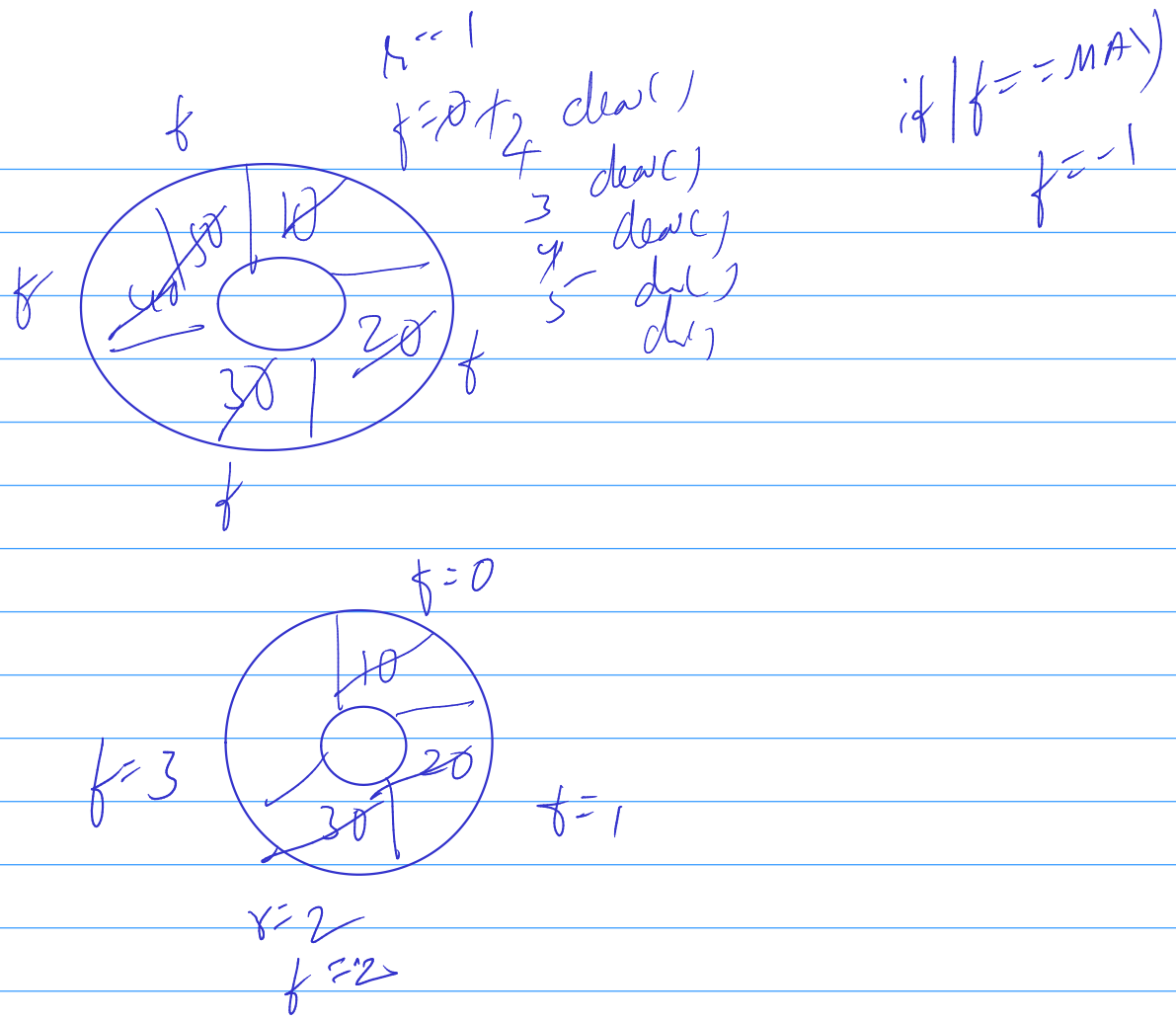
f=4

f=3

Deq()

f==MAX && r>-1

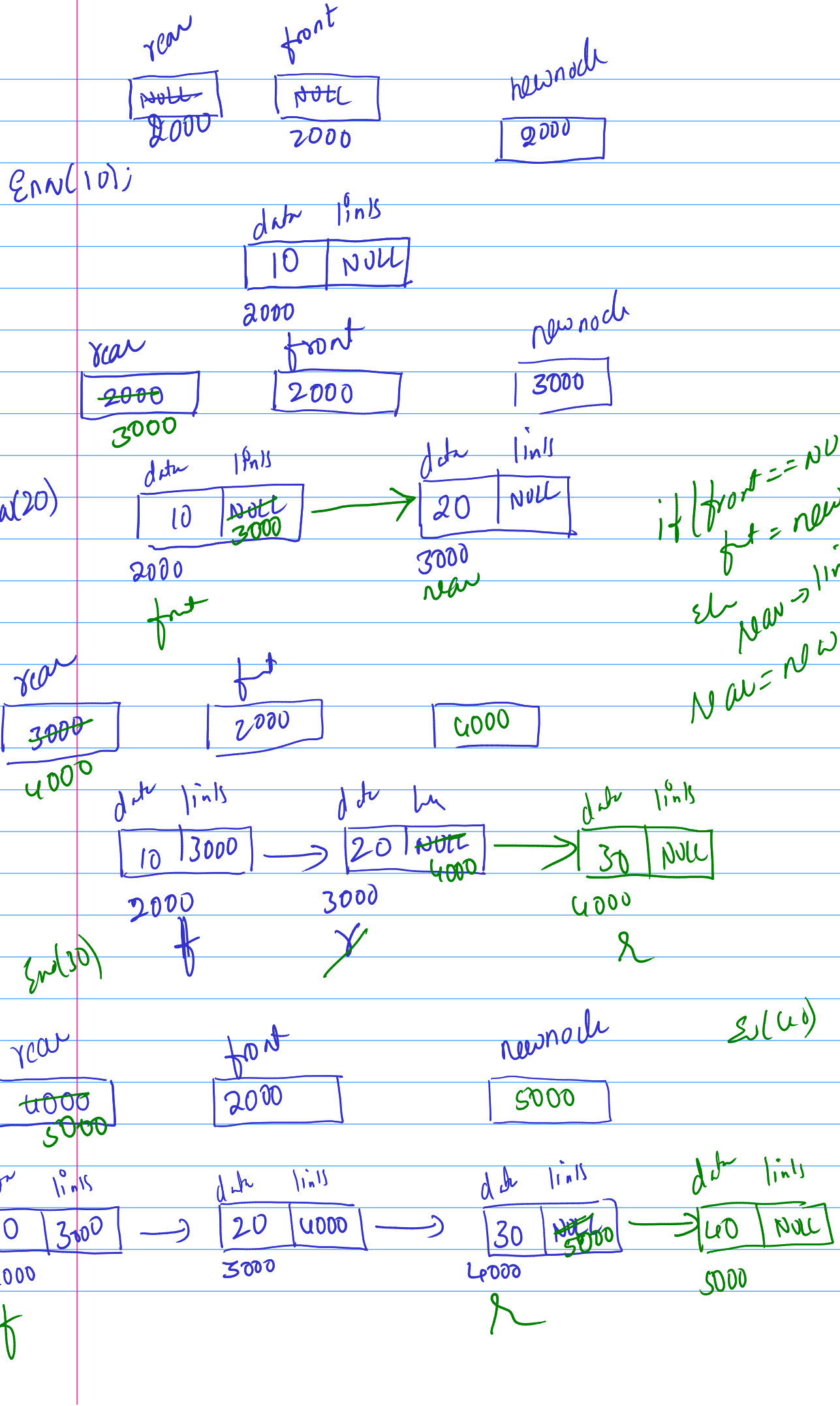
f=0

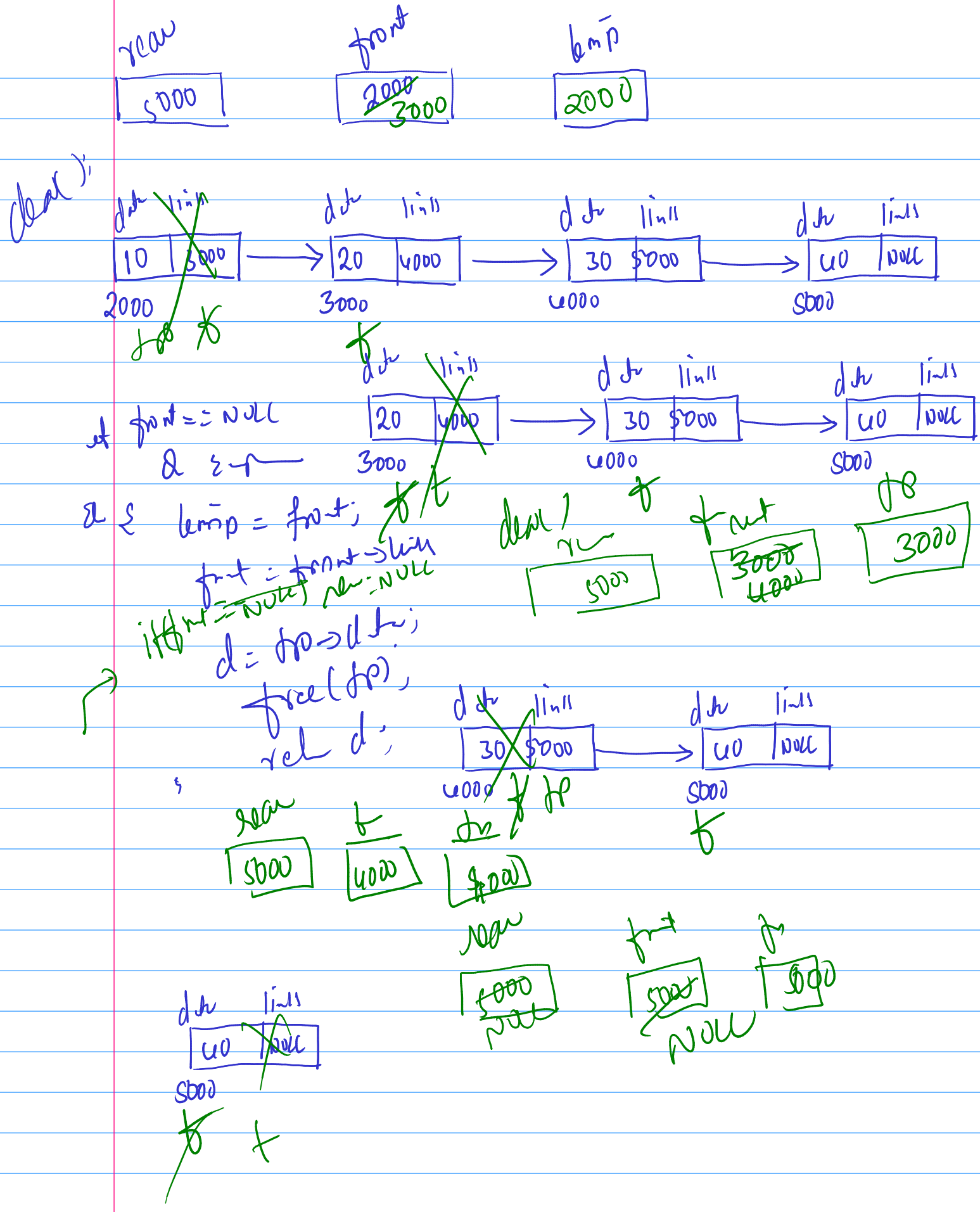


Queue with link list

rear points last inserted node .
rear is a pointer to last node

front points to first node
front is a pointer to first node
enq means add at rear , i.e addlast
deq means delete from front, i.e delete first


rear
front
newnode
NULL
2000
Enq(10);
data
links
10
NULL
3000
Enq(20)
20
4000
Enq(30)
30
5000
Enq(40)
40
if(front==NULL)
front=newnode
else
rear->links=newnode
rear=newnode
f
r


rear
5000
front
2000
3000
temp
2000
del();
data
link
10
3000
20
4000
30
5000
40
NULL
2000
3000
4000
5000
if front == NULL
temp = front;
front = front->link
if(front == NULL) rear = NULL
d = temp->data;
free(temp);
return d;
3000
4000
5000
NULL